

Homeopathic Assistant

Desktop Application for Homeopathic Practitioners

Database Systems — Project Documentation

Python · PyQt6 · MySQL

May 2026

1. What Is This Project?

Homeopathic Assistant is a desktop application that helps a homeopathic doctor manage their clinic. It has three main jobs:

- **Store patient records** — name, age, symptoms, medical history, constitutional type, and so on.
- **Find the right remedy** — the doctor enters a patient's symptoms, and the app searches Kent's Repertory (a 100-year-old medical reference book) to return the best-matching remedies ranked by score.
- **Browse and reference** — the doctor can browse all remedies in the repertory, look up diseases, and view consultation history.

All data is stored in a MySQL database. The interface is built with Python and PyQt6 (a desktop UI framework). There is no internet connection needed — everything runs locally.

2. How We Built It — Step by Step

We divided the project into clear milestones, completing one before starting the next.

1 Plan the Database
We read through Kent's Repertory to understand its structure. It has chapters (e.g. MIND, CHEST), rubrics (symptom phrases), and remedies listed under each rubric with a grade. We designed 8 tables to hold all of this plus patient data.

2 Create the Database Schema
We wrote the SQL schema — all 8 tables with correct column types, primary keys, foreign keys, and a FULLTEXT index on the rubrics table so keyword search would be fast.

3 Import Kent's Repertory
We wrote `import_kent.py` to parse the `kent.txt` file line by line. Chapter headings go into the `chapters` table, indented symptom lines go into `rubrics`, and remedy tokens (like Bell., Sulph.) are linked in `rubric_remedy`. After the main import we ran `extract_remedy_links.py` to catch any missed links. This is a one-time setup step.

4 Build the Service Layer
We wrote `PatientService` (add, edit, delete, search patients) and `RemedyService` (symptom matching, rubric browsing, disease lookups) as separate Python classes. No SQL appears in the interface code — all queries are in these service classes.

5 Build the User Interface
We used PyQt6 to build a tabbed desktop window: Add Patient, Patient Records, Remedy Browser, and Disease Browser. A central theme file (`theme.py`) controls all colours and styling. Dialogs handle viewing, editing, and remedy assignment.

6

Connect Everything and Test

We connected the UI to the services, tested adding patients, running symptom searches, and browsing remedies. We fixed a case-sensitivity bug in remedy lookups (Bell. vs bell.) and tuned the scoring formula to match how practitioners actually use the repertory.

3. Database Schema

The database is called **homoeo**. It has 8 tables split into two groups: 4 tables for the repertory data and 4 for clinic management.

Repertory Tables (populated by the import script)

Table	What It Stores
chapters	The main sections of Kent's Repertory — MIND, HEAD, CHEST, ABDOMEN, etc.
rubrics	Every symptom phrase (rubric) with its full path, level in hierarchy, and parent. Has a FULLTEXT index for fast keyword search.
remedies	All homeopathic remedies with their abbreviation (e.g. sulph.), full Latin name, and thermal type (HOT/COLD).
rubric_remedy	Junction table linking each rubric to the remedies listed under it, plus a grade (1, 2, or 3) showing how strongly the remedy is indicated.

Clinic Management Tables

Table	What It Stores
patients	Full patient records — name, age, gender, symptoms, thermal type, miasm, medical history, aggravation, amelioration, and more.
patient_visits	A record of each visit date linked to the patient. Created automatically when a remedy is saved.
disease_remedies	A curated list of common diseases with their standard remedies, key symptoms, and clinical notes. Powers the Disease Browser tab.
remedy_relations	Relationships between remedies — complementary pairs, antidotes, and remedies that should not be used together.

Important Column Details

rubric_remedy — the most important table for remedy matching:

Column	Type	Meaning
rubric_id	INT FK → rubrics.id	Which rubric
remedy_id	INT FK → remedies.id	Which remedy
grade	TINYINT (1, 2, or 3)	1 = plain text in Kent, 2 = italic, 3 = bold (strongest)

patients — key fields:

Column	Type
patient_id	VARCHAR(20) — e.g. P1715867234 (auto-generated from timestamp)
name	VARCHAR(100) NOT NULL
thermal_type	VARCHAR(10) — HOT or COLD
miasm	VARCHAR(20) — Psora / Sycosis / Syphilis / Tubercular

Column	Type
mental_symptoms / body_issues / modalities / aggravation / amelioration	TEXT fields storing the patient's full symptom picture

4. Entity Relationship Diagram (ERD)

The diagram below shows all 8 tables and how they connect. Solid lines are foreign key relationships. Dashed lines are logical links through abbreviation fields.

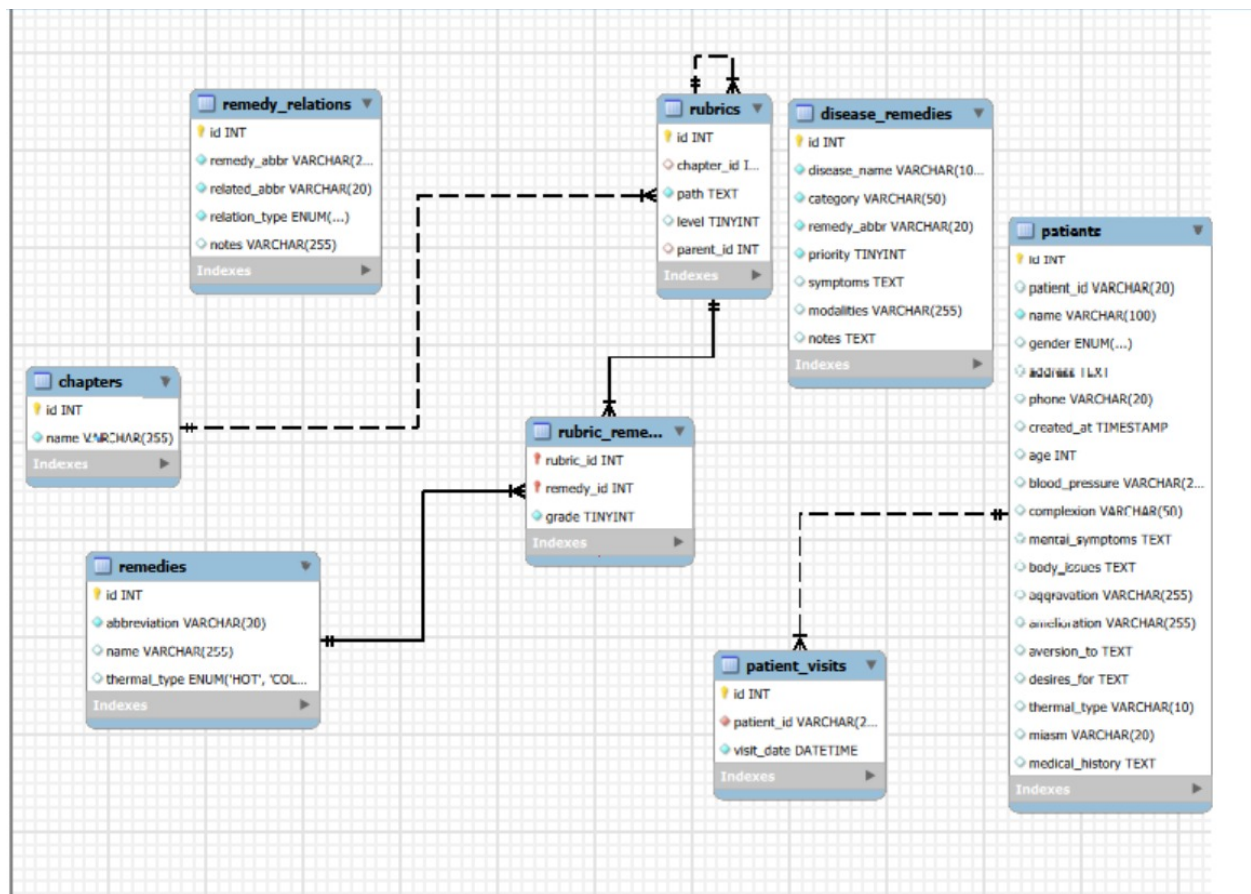


Figure 1 — homoeo database ERD

Relationship	Meaning
chapters → rubrics	One chapter has many rubrics
rubrics → rubrics (self)	A rubric can have a parent rubric (hierarchy)
rubrics → rubric_remedies	One rubric links to many remedies through the junction table
remedies → rubric_remedies	One remedy appears in many rubrics through the junction table
patients → patient_visits	One patient can have many visit records

5. How Remedy Matching Works

This is the core feature of the application. When the doctor enters a patient's symptoms and clicks Find Remedies, the following happens:

Step 1 — Search rubrics

For each symptom typed (e.g. "anxiety", "burning pains"), the app runs a MySQL FULLTEXT search on the rubrics table. MySQL scores each matching rubric by how well it matches the keyword and how deep it is in the hierarchy.

Step 2 — Score remedies

The app takes all the matching rubric IDs and does a database JOIN with rubric_remedy to find every remedy linked to those rubrics. Each remedy's score is calculated as:

$$\text{Remedy Score} = \text{Rubric Relevance} \times \text{Grade}$$

A grade-3 remedy counts 3× more than a grade-1 remedy — just like how a practitioner reads the book.

Step 3 — Filter and rank

If the doctor selected HOT or COLD thermal type, only matching remedies pass through (universal remedies like Sulphur always appear). Results are sorted by score, converted to a percentage, and colour-coded:

Colour	Score	Meaning
Green	70% and above	Strong match — highly indicated
Yellow	40% – 69%	Moderate match — consider this remedy
Red	Below 40%	Weak match — occasionally indicated

6. Application Screens

Screen / Tab	What It Does
Add Patient	Form to register a new patient. Sections: Basic Info, Medical & Constitutional, Symptoms. A unique ID is created automatically.
Patient Records	Searchable list of all patients. Each row has View, Remedies, Edit, and Delete buttons.
Remedy Browser	Browse all remedies from Kent's Repertory. Shows grade (★★★ / ★★ ■ / ★ ■ ■) and all rubrics a remedy appears in.
Disease Browser	Browse diseases by category (Mental, Joints, Fever, Skin, etc.) and see standard remedies with key symptoms and notes.

7. How to Set Up and Run

Step	What to Do
1	Install Python 3.8+, MySQL 8, then run: <code>pip install -r requirements.txt</code>

Step	What to Do
2	Create the database: <code>CREATE DATABASE homoeo;</code> then run the schema SQL file.
3	Put your MySQL credentials in <code>config.py</code>
4	Place <code>kent.txt</code> in the project folder.
5	Run: <code>python scripts/import_kent.py</code> (imports the full repertory – takes a few minutes).
6	Run: <code>python scripts/extract_remedy_links.py</code> (fixes any missed remedy links).
7	Start the app: <code>python main.py</code> or double-click <code>run.bat</code> on Windows.

End of Documentation — Homeopathic Assistant